

수업 03 · 전체 과정 7/20일차

Python 심화

2026-10-15 (목) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	5
1교시 · 반복 업무와 함수	6
2교시 · 함수 입력과 반환	12
3교시 · 파일을 읽고 결과 저장	19
4교시 · 오류를 단서로 읽기	27
5교시 · 검사 규칙과 재확인	33
6교시 · 로그 검사기와 사용 안내	39
용어와 코드 기호 찾아보기	51
오늘 배운 내용 확인하기	51
파일과 다음 수업	54

오늘 준비할 것

Python 심화 · 함수와 파일 검사

2026-10-15 · Python 심화

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

원본은 그대로 두고 work 폴더의 지정한 파일을 수정하세요. 계정이나 실제 장비 없이 합성 자료로 진행합니다.

오늘의 연결

어제 읽은 조건과 반복을 함수 안에 묶고 파일을 처리하는 데 사용합니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Python 실행기는 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data-code-work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장 후 정확한 work 파일을 실행하거나 강사 출력·제공 예와 수동 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치 문제로 수업을 멈추지 말고 제공한 전체 코드·출력과 동일 입력을 사용해 줄별 값을 직접 추적합니다. 종이에도 같은 답칸을 작성합니다. 개인 실행은 강사가 환경을 준비한 뒤 확인합니다.

[교안](#) · [실습](#) · [용어](#)

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

함수의 입력과 결과

질문: 함수 안에서 print만 했다면 호출한 곳이 그 숫자를 계산에 쓸 수 있나요?
풀이: 화면 표시와 return으로 돌려주는 결과를 구분합니다. 호출 결과를 다른 계산에 쓰려면 필요한 값을 돌려줘야 합니다.

파일 읽기

질문: CSV 칸에 28.0이 적혀 있으면 자동으로 float인가요?
풀이: 일반적인 CSV 읽기는 문자열 값을 줍니다. 공백·누락·변환 실패를 확인한 뒤 숫자로 바꿉니다.

미측정

질문: 빈 온도를 0으로 바꾸면 평균은 어떻게 되나요?
풀이: 측정하지 않은 기록이 실제 0도처럼 계산되어 평균을 낮춥니다. 측정 여부와 정상 숫자를 따로 기록합니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	함수의 정의와 호출	A01, A02, A03
2교시 11:00~11:50	화면 출력과 반환	A04, A05, A06
3교시 13:00~13:50	파일 입력과 결과 저장	A07, A08, A09
4교시 14:00~14:50	오류를 단서로 확인하기	A10, A11, A12
5교시 15:00~15:50	허용 범위와 수정 근거	A13, A14, A15
6교시 16:00~16:50	로그 검사기의 전체 흐름	A16, A17, A18

설명·시연·함께 수행·개별 수행·풀이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

2026-10-15 · Python 심화

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

확정 운영은 50분×6교시·순수 수업 300분이며 중식은 11:50~13:00입니다. 실제 학습자 리허설 전입니다. 원본을 보존하고 work의 지정한 부분을 수정합니다. 계정이나 실제 장비 없이 합성 자료로 진행합니다.

1교시 · 반복 업무와 함수

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 함수의 정의와 호출

먼저 할 활동: A01, A02, A03

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 같은 업무를 함수로 묶는 이유

사례로 이해하기 · 작업의 입력·처리·결과를 함수 계약으로 쓰기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

안내 데스크는 방문자 이름을 받아 같은 환영 문장을 만듭니다. 이름마다 문장을 복사하면 한 문구를 고칠 때 여러 곳을 찾아야 합니다. 함수는 공통 절차를 이름 붙여 묶는 방법입니다. 먼저 코드를 적기 전에 ‘이름 문자열을 받아 환영 문장 문자열을 돌려준다’라는 계약을 씁니다. 입력은 호출자가 건네는 값, 처리는 문장을 만드는 일, 결과는 호출자가 다시 사용할 값입니다. 이 세 역할이 분명해야 함수를 어디까지 묶을지 판단할 수 있습니다.

매개변수는 받을 값의 자리 이름이고 인자는 실제 호출에서 건네는 값입니다. 신청서의 이름 칸과 그 칸에 적는 민수라는 글의 관계로 생각합니다. 아래 완결 예를 work/E01_greeting.py라는 새 파일로 저장하고 course 폴더에서 `python -X utf8 work/E01_greeting.py`를 실행합니다.

```
def make_greeting(name):
    return "안녕하세요, " + name

first = make_greeting("민수")
second = make_greeting("지아")
print(first)
print(second)
```

첫 줄의 def는 작업을 정의하고 괄호 안 name은 받을 자리입니다. 콜론 뒤 들여쓴 둘째 줄은 호출 때 문자열을 연결하고 return으로 돌려줍니다. 셋째 줄은 “민수”를 주고 받은 문장을 first에 연결합니다. 넷째 줄은 별도 호출에 “지아”를 주고 second에 연결합니다. 마지막 두 줄은 각 결과를 표시합니다. 예상 출력은 ‘안녕하세요, 민수’와 ‘안녕하세요, 지아’ 두 줄입니다. 정의 자체에서는 인자가 출력되지 않습니다.

두 호출은 같은 작업을 쓰지만 다른 입력으로 다른 결과를 얻습니다. 함수 안 name이 항상 민수라고 외우면 둘째 호출을 잘 못 읽습니다. 들여쓰기가 빠지면 함수 본문의 경계가 깨져 오류가 날 수 있으므로 둘째 줄 앞 공백을 확인합니다. 이름을 숫자로 주면 문자열 연결이 실패할 수 있으므로 오늘 계약은 문자열 입력만 다룹니다. 실제 계정 조회나 메시지 전송은 하지 않습니다.

새 상황에 독립적으로 적용하기

복습·도전: 물품 라벨에 상품명과 보관함을 받아 '장갑 / B02'처럼 한 문자열을 만드는 작업을 설계하세요. 입력 자리 두 개, 반환할 종류, 순서를 바꿨을 때 생길 문제를 문장으로 쓰세요. 첫 입력은 장갑/B02, 둘째 입력은 마스크/A01입니다. 이 두 호출의 결과를 예상하세요.

정답과 오답의 이유

입력은 상품명·보관함 문자열, 결과는 두 내용을 순서대로 연결한 문자열입니다. 장갑/B02와 마스크/A01의 결과는 각 입력에 따라 달라집니다. 순서를 바꾸면 B02 / 장갑처럼 업무 양식이 달라집니다. 컴퓨터가 이름의 뜻을 추측해 입력 위치를 고쳐 준다는 답은 오답입니다.

핵심 세 질문과 해설

1. **정의하면 바로 실행되나요?** 본문은 해당 함수를 호출했을 때 실행됩니다.
2. **매개변수와 인자는 같은 이름인가요?** 매개변수는 받을 자리, 인자는 호출에서 준 실제 값입니다.
3. **어떤 코드를 함수로 묶나요?** 공통 작업과 입력·결과 계약이 분명한 부분을 먼저 묶습니다.

개념. 함수는 한 가지 일을 이름 붙여 묶은 코드입니다. 매번 같은 안내 절차를 읽는 대신 '온도 상태 확인'이라는 작업을 정해 두고 필요한 값으로 부를 수 있습니다. 무엇을 입력하고 무엇을 결과로 돌려줄지 먼저 정합니다.

왜 필요한가. 같은 조건식을 여러 곳에 복사하면 한 곳만 수정하고 다른 곳을 놓칠 수 있습니다. 반복되는 규칙을 모으면 변경 위치를 찾기 쉽습니다. 첫 코드부터 모든 줄을 함수로 만들 필요는 없습니다.

함께 보는 예. 온도 22.5·25.0·24.0을 각각 24.0과 비교합니다. 공통 작업은 비교하여 상태를 정하는 과정이며 달라지는 것은 입력 온도입니다. 같은 업무 이름에 다른 입력을 줄 수 있습니다.

헷갈리는 경우. 이름이 check라고 해서 검증을 모두 수행하는 것은 아닙니다. 본문이 실제로 어떤 조건을 보는지 읽어야 합니다. 함수 이름은 약속을 설명하지만 정확성을 보장하지 않습니다.

제공 입력

세 업무: 22.5와24.0 비교 / 25.0과24.0 비교 / 24.0과24.0 비교.
 답칸: 공통 작업 / 달라지는 입력 / 결과 / 이름 제안.

A01 · 따라 하기와 작성칸

A01 · 같은 업무를 함수로 묶는 이유

목적: 같은 조건식을 여러 곳에 복사하면 한 곳만 수정하고 다른 곳을 놓칠 수 있습니다. 반복되는 규칙을 모으면 변경 위치를 찾기 쉽습니다. 첫 코드부터 모든 줄을 함수로 만들 필요는 없습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A01_input.md](#)
- [A01_notes.md](#)

단계

1. 같은 부분과 다른 값을 구별합니다.
2. 공통 작업에 temperature_status라는 이름을 붙입니다.
3. 받을 값과 결과의 뜻을 문장으로 씁니다.
4. 기준 변경 때 수정할 위치를 비교합니다.

예상 결과: 공통 규칙·온도 입력·상태 결과를 구별합니다.

확인 방법: 줄 수가 짧으면 함수가 필요 없다고 단정할 수 있나요?

오류 해결: 입력 카드 한 장을 받고 상태 카드 한 장을 돌려주는 역할극으로 그림니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A02 · def로 정의하고 이름으로 호출하기

개념. def는 함수를 정의하는 문장입니다. 이름 뒤 괄호와 콜론을 쓰고 들여쓴 줄을 함수 본문으로 묶습니다. 정의만 하면 본문은 아직 실행되지 않으며 이름을 괄호와 함께 불러야 합니다.

왜 필요한가. 위에서 아래로 읽는 원리는 유지하지만 함수 본문은 호출할 때 실행됩니다. 정의 위치와 실제 실행 시점을 나누어 보면 출력 순서가 이해됩니다.

함께 보는 예. show_notice를 정의한 뒤 준비를 출력합니다. 함수 호출 때 안내가 나오고 호출에서 돌아온 뒤 끝이 나옵니다. 함수 본문의 줄 번호가 위에 있다고 가장 먼저 출력되는 것은 아닙니다.

헷갈리는 경우. show_notice라고 이름만 적는 것과 show_notice()로 호출하는 것은 다릅니다. 괄호는 실제 작업을 요청하는 표기입니다.

전체 코드

```
def show_notice():
    print("점검을 시작합니다")

print("준비")

show_notice()

print("끝")
```

각 줄의 뜻

줄	코드	뜻
1	<code>def show_notice():</code>	안내하는 작업을 정의합니다.
2	<code>print("점검을 시작합니다")</code>	호출할 때 실행할 출력입니다.
3	<code>print("준비")</code>	바깥에서 준비를 출력합니다.
4	<code>show_notice()</code>	정의한 함수를 호출합니다.
5	<code>print("끝")</code>	돌아온 뒤 끝을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A02_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A02_example.py
```

원본 코드의 예상 출력:

```
준비
점검을 시작합니다
끝
```

작은 수정: `show_notice()` `print("끝")`를 `show_notice()` `show_notice()` `print("끝")`로 바꿉니다.

수정 후 예상 출력:

```
준비
점검을 시작합니다
점검을 시작합니다
끝
```

A02 · 따라 하기와 작성칸

A02 · def로 정의하고 이름으로 호출하기

목적: 위에서 아래로 읽는 원리는 유지하지만 함수 본문은 호출할 때 실행됩니다. 정의 위치와 실제 실행 시점을 나누어 보면 출력 순서가 이해됩니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)
- [A02_example.py](#)
- [A02_notes.md](#)
- [A02_example.py](#)

단계

1. 정의와 호출을 표시합니다.
2. 실제 출력 순서를 씁니다.
3. 호출을 한 번 더 넣어 예상합니다.
4. 정의 횟수와 본문 실행 횟수를 설명합니다.

예상 결과: 정의 한 번·호출 두 번이면 안내 출력 두 번.

확인 방법: 정의만 하고 호출하지 않으면 본문 출력은 나오나요?

오류 해결: 본문 출력이 예상과 다르면 print가 함수 안에 들여써져 있는지 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A03 · 매개변수와 실제 인자

개념. 매개변수는 받을 값의 이름이고 인자는 호출할 때 실제로 주는 값입니다. 양식의 ‘장소’ 칸과 거기에 넣는 ‘입구’라는 내용의 관계에 비유할 수 있습니다.

왜 필요한가. 고정 값을 함수 안에서 매번 바꾸는 대신 호출값을 달리해 같은 작업을 사용합니다. 함수 안 이름이 어디서 값을 받는지 알아야 전달한 값을 추적할 수 있습니다.

함께 보는 예. sensor_id와 value는 받을 이름입니다. 첫 호출에서 S01·22.5가 연결되고 다음 호출에서는 S02·25.0이 연결됩니다. 각 호출은 서로 다른 값으로 같은 본문을 사용합니다.

헷갈리는 경우. 인자의 순서를 바꾸어도 컴퓨터가 업무 의미를 추측해 바로잡지 않습니다. 센서명 위치에 숫자를 주면 잘못된 위치의 값이 출력될 수 있습니다.

전체 코드

```
def show_temperature(sensor_id, value):
    print(sensor_id, value)

show_temperature("S01", 22.5)

show_temperature("S02", 25.0)
```

각 줄의 뜻

줄	코드	뜻
1	<code>def show_temperature(sensor_id, value):</code>	센서명과 값을 받는 함수를 정의합니다.
2	<code>print(sensor_id, value)</code>	이번 호출에서 받은 값을 출력합니다.
3	<code>show_temperature("S01", 22.5)</code>	첫 센서와 온도를 전달합니다.
4	<code>show_temperature("S02", 25.0)</code>	둘째 센서와 온도를 전달합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A03_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A03_example.py
```

원본 코드의 예상 출력:

```
S01 22.5
S02 25.0
```

작은 수정: `show_temperature("S02", 25.0)`를 `show_temperature("S03", 0.0)`로 바꿉니다.

수정 후 예상 출력:

```
S01 22.5
S03 0.0
```

A03 · 따라 하기와 작성칸

A03 · 매개변수와 실제 인자

목적: 고정 값을 함수 안에서 매번 바꾸는 대신 호출값을 달리해 같은 작업을 사용합니다. 함수 안 이름이 어디서 값을 받는 지 알아야 전달한 값을 추적할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)

- [A03_example.py](#)

- [A03_notes.md](#)
- [A03_example.py](#)

단계

1. 전달값과 받을 이름을 짝짓습니다.
2. 둘째 입력을 S03·0.0으로 바꿉니다.
3. 값 하나가 부족한 호출과 정상 호출을 비교합니다.
4. 함수 내부를 고치지 않고 결과가 달라지는 이유를 씁니다.

예상 결과: 둘째 출력 S03 0.0. 인자 부족은 TypeError.

확인 방법: value는 모든 호출에서 같은 값을 뜻하나요?

오류 해결: 받을 값 개수와 전달한 값 개수·순서를 비교합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

2교시 · 함수 입력과 반환

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 화면 출력과 반환

먼저 할 활동: A04, A05, A06

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · print와 return의 차이

사례로 이해하기 · 화면 표시와 다음 계산의 반환값 구별하기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

청구서 담당자는 수량 3과 단가 1200원으로 상품 금액을 계산한 뒤 배송비 2000원을 더합니다. 첫 계산이 화면에 3600을 보여 주었다고 다음 계산에서 그 값을 사용할 수 있는 것은 아닙니다. 화면 표시는 사람에게 보여 주는 행동이고 반환은 함수를 부른 곳에 결과 값을 보내는 행동입니다. 전자계산기에서 값을 읽는 것과 그 값을 다음 식의 입력으로 전달하는 것을 구별합니다. 필요한 최종 결과를 먼저 쓰면 print와 return의 역할이 선명해집니다.

함수의 입력 계약은 단가와 수량이라는 정수 두 개, 반환 계약은 상품 금액 정수입니다. 수업용 처리 규칙은 단가×수량이며 세금이나 할인은 넣지 않습니다. 호출에서 1200과 3을 주면 함수 안에서 3600을 만들고 return으로 보냅니다. 호출한 쪽

은 받은 값에 2000을 더해 최종 5600원을 얻습니다. 손 추적표는 입력 1200·3 → 함수 계산 3600 → 반환 3600 → 배
송비 포함 5600 순서입니다. 화면에 무엇이 보이는가와 변수에 무엇이 연결되는가를 다른 칸으로 적습니다.

print만 하는 함수는 3600을 화면에 보이게 해도 별도 return이 없으면 호출 결과가 None입니다. None에 2000을 더하
면 숫자 계산이 되지 않습니다. 오류를 피하려고 None을 0으로 바꾸면 상품 금액이 사라져 잘못된 청구서가 됩니다. 결과를
계산하는 함수에 return을 두고 필요한 표시를 호출한 쪽에서 맡기는 구조를 생각합니다. 또한 return을 만나는 즉시 그 호
출은 끝납니다. 그 아래 줄은 같은 호출에서 이어서 실행되지 않습니다. 조건이 여러 갈래인 함수라면 각 갈래가 무엇을 반환
하는지 읽어야 합니다. 한 갈래에서만 반환하면 나머지 입력에서 뜻밖의 None이 생길 수 있습니다.

새 상황에 독립적으로 적용하기

복습·도전: 배송 준비 함수는 포장비 500원을 화면에 표시하지만 반환문이 없습니다. 호출 결과를 fee에 연결한 뒤 총액
3000에 더하려 합니다. 화면값·fee 값·다음 계산을 분리해 설명하고 올바른 역할 배치를 문장으로 설계하세요.

정답과 오답의 이유

화면에는 500이 보이지만 fee는 None이며 다음 덧셈은 TypeError가 날 수 있습니다. 함수가 500을 반환하고 호출자가
총액에 더한 뒤 표시하도록 설계합니다. 화면에 500이 보이므로 fee도 500이라는 답은 표시와 반환을 혼동한 것입니다.

핵심 세 질문과 해설

1. **print의 결과가 자동 저장되나요?** 표시와 반환은 별도이며 호출 결과를 보려면 return을 확인합니다.
2. **return 뒤 안내가 실행되나요?** 같은 호출은 return에서 끝나므로 아래 줄까지 가지 않습니다.
3. **None을 0으로 고쳐도 되나요?** 업무 결과가 누락된 원인을 고쳐야 하며 임의 0은 계산 의미를 바꿉니다.

개념. print는 값을 화면에 보여 주고 return은 함수를 부른 위치로 값을 돌려줍니다. 화면에 결과가 보였다는 사실과 다음
계산에 사용할 결과를 받았다는 사실은 다릅니다.

왜 필요한가. 결과를 JSON에 저장하거나 집계하려면 함수가 값을 돌려줘야 합니다. 화면 글자를 다른 코드가 저절로 변수
에 담지는 않습니다.

함께 보는 예. add_offset은 받은 두 수를 더해 반환합니다. result가 23.5를 받고 다음 계산에서 사용합니다. 더하는 값은
수업 계산 예이며 실제 장비 교정값으로 사용하지 않습니다.

헛갈리는 경우. return 없이 print만 하는 함수의 반환값은 None입니다. 그 값을 숫자처럼 더하려고 하면 오류가 날 수 있습
니다. 화면만 보지 말고 return의 역할을 확인합니다.

전체 코드

```
def add_offset(value, offset):
    return value + offset

result = add_offset(22.5, 1.0)

print(result)

print(result + 1)
```

각 줄의 뜻

줄	코드	뜻
1	<code>def add_offset(value, offset):</code>	두 값을 받는 작업을 정의합니다.
2	<code>return value + offset</code>	합을 호출한 곳으로 돌려줍니다.
3	<code>result = add_offset(22.5, 1.0)</code>	돌아온 결과를 result에 연결합니다.
4	<code>print(result)</code>	받은 결과를 출력합니다.
5	<code>print(result + 1)</code>	결과를 다음 계산에 사용합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A04_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A04_example.py
```

원본 코드의 예상 출력:

```
23.5
24.5
```

작은 수정: 22.5, 1.0를 22.5, 2.0로 바꿉니다.

수정 후 예상 출력:

```
24.5
25.5
```

A04 · 따라 하기와 작성칸

A04 · print와 return의 차이

목적: 결과를 JSON에 저장하거나 집계하려면 함수가 값을 돌려줘야 합니다. 화면 글자를 다른 코드가 저절로 변수에 담지는 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A04_input.md](#)

- [A04_example.py](#)
- [A04_notes.md](#)
- [A04_example.py](#)

단계

1. 출력과 반환의 경로를 구별합니다.
2. result가 값을 받는 위치를 찾습니다.
3. offset을 2.0으로 바꿔 예상합니다.
4. 두 출력의 관계를 확인합니다.

예상 결과: 24.5를 반환하고 다음 계산은 25.5.

확인 방법: 화면에 보인 값은 언제나 같은 함수의 반환값인가요?

오류 해결: None이 보이면 return이 있는지, 특정 경로에서 반환 없이 끝나는지 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과-근거: _____
 다음에 확인할 것: _____

A05 · 조건을 담은 상태 함수

개념. 함수 안에서도 if를 사용할 수 있습니다. 입력을 받아 조건에 맞는 상태 문자열을 돌려주는 규칙을 만듭니다. return을 만나면 그 호출은 결과를 돌려주고 끝납니다.

왜 필요한가. 상태 규칙과 출력 위치를 나누면 같은 결과를 화면이나 파일에 사용할 수 있습니다. 함수가 입력 자체를 수정하는지 결과만 만드는지도 계약으로 구별합니다.

함께 보는 예. None이면 미측정, 24.0 초과면 확인, 나머지는 정상입니다. 앞의 return을 실행한 호출은 아래 비교로 다시 내려가지 않습니다.

헷갈리는 경우. 함수로 묶었다고 자료형 규칙이 없어지는 것은 아닙니다. None을 먼저 숫자와 비교하면 TypeError가 나므로 값 없음의 처리를 앞에 둡니다.

전체 코드

```
def temperature_status(value):
    if value is None:
        return "미측정"
    if value > 24.0:
        return "확인"
    return "정상"

print(temperature_status(22.5))
print(temperature_status(None))
print(temperature_status(25.0))
```

각 줄의 뜻

줄	코드	뜻
1	<code>def temperature_status(value):</code>	값을 받는 함수를 정의합니다.
2	<code>if value is None:</code>	값 없음을 먼저 확인합니다.
3	<code>return "미측정"</code>	미측정으로 반환하고 끝냅니다.
4	<code>if value > 24.0:</code>	숫자의 기준 초과를 묻습니다.
5	<code>return "확인"</code>	초과 상태를 반환합니다.
6	<code>return "정상"</code>	나머지 상태를 반환합니다.
7	<code>print(temperature_status(22.5))</code>	정상 입력을 확인합니다.
8	<code>print(temperature_status(None))</code>	미측정 입력을 확인합니다.
9	<code>print(temperature_status(25.0))</code>	초과 입력을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A05_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A05_example.py
```

원본 코드의 예상 출력:

정상
미측정
확인

작은 수정: temperature_status(25.0)를 temperature_status(24.0)로 바꿉니다.

수정 후 예상 출력:

정상
미측정
정상

A05 · 따라 하기와 작성칸

A05 · 조건을 담은 상태 함수

목적: 상태 규칙과 출력 위치를 나누면 같은 결과를 화면이나 파일에 사용할 수 있습니다. 함수가 입력 자체를 수정하는지 결과만 만드는지도 계약으로 구별합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A05_input.md](#)
- [A05_example.py](#)
- [A05_notes.md](#)
- [A05_example.py](#)

단계

1. 입력별 도달하는 return을 표시합니다.
2. 마지막 입력을 24.0으로 바꿉니다.
3. 상태 문자열만 생기는지 입력이 바뀌는지 설명합니다.
4. None 확인을 먼저 하는 이유를 씁니다.

예상 결과: 정상·미측정·정상. 같은 값은 초과가 아닙니다.

확인 방법: return 뒤의 코드를 같은 호출이 모두 실행하나요?

오류 해결: 미측정 오류는 숫자 비교보다 None 검사가 앞인지 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A06 · 예상 사례표로 함수 확인하기

개념. 검증 사례는 입력과 기대 결과를 미리 적어 실제 결과와 비교하는 한 경우입니다. 정상값뿐 아니라 경계와 값 없음의 사례도 포함합니다.

왜 필요한가. 프로그램이 보여 준 결과를 그대로 정답으로 적으면 잘못된 코드도 통과합니다. 실행 전에 업무 규칙에서 기대값을 정해야 합니다.

함께 보는 예. 23.9·24.0·24.1은 정상·정상·확인입니다. None은 미측정이고 0.0은 실제 숫자입니다. 다섯 사례는 서로 다른 실수를 확인합니다.

헷갈리는 경우. 다섯 사례를 통과했다고 모든 입력을 확인한 것은 아닙니다. 글자와 범위 밖 숫자는 아직 검사하지 않았다는 한계를 남깁니다.

제공 입력

```
입력:23.9 /24.0 /24.1 /None /0.0.
각 행: 기대 상태 / 실제 상태 / 일치 여부 / 이 사례가 필요한 이유.
```

A06 · 따라 하기와 작성칸

A06 · 예상 사례표로 함수 확인하기

목적: 프로그램이 보여 준 결과를 그대로 정답으로 적으면 잘못된 코드도 통과합니다. 실행 전에 업무 규칙에서 기대값을 정해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A06_input.md](#)

- [A06_notes.md](#)

단계

- 24.0 초과 규칙을 문장으로 적습니다.
- 다섯 기대값을 먼저 채웁니다.
- A05 호출값을 바꾸어 대조합니다.
- 통과한 범위와 남은 입력 종류를 나눕니다.

예상 결과: 정상/정상/확인/미측정/정상과 판단 이유.

확인 방법: 20과 30만 확인하면 놓칠 수 있는 실수는 무엇인가요?

오류 해결: 기대값이 애매하면 코드 대신 초과라는 업무 문구를 먼저 읽습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 파일을 읽고 결과 저장

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 파일 입력과 결과 저장

먼저 할 활동: A07, A08, A09

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 파일 경로와 텍스트 읽기

사례로 이해하기 · 읽은 텍스트에서 값으로, 값에서 저장 파일로

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

점검 담당자는 CSV 입력에서 온도를 읽고 요약 JSON을 남깁니다. 파일은 디스크에 저장된 내용이고 읽어 온 값은 프로그램이 처리하는 내용입니다. 화면에 요약을 출력했다는 사실만으로 결과 파일이 저장되지는 않습니다. 흐름을 파일 주소 → 읽은 원문 → 해석한 값 → 결과 묶음 → 저장 주소 순서로 그립니다. 같은 내용처럼 보이는 단계도 역할과 저장 종류가 달라 지므로 경계를 하나씩 확인합니다. 원자료와 결과 파일은 서로 다른 주소로 정합니다.

예를 들어 헤더 `sensor_id,temperature_c` 아래 `S08,20.5` 한 행이 있습니다. DictReader 같은 CSV 읽기 도구는 헤더 를 이름표로 삼고 기본 셀값을 문자열로 읽습니다. 원문 "20.5"를 실수 20.5로 해석한 다음 요약의 측정수 1과 평균 20.5 를 만듭니다. JSON으로 표현한 완성 예는 `{"count":1,"average":20.5}`입니다. 숫자에 따옴표를 붙이지 않았으므로 JSON에서는 숫자값입니다. 원문 CSV에 숫자처럼 적혀 있다는 사실과 읽은 Python 값의 종류를 구별합니다.

경로는 파일의 주소이며 현재 터미널 위치와 코드 파일 위치가 같다고 가정하지 않습니다. 기존 예제는 코드 위치를 기준으로 `data`와 `work`를 찾습니다. 폴더 배치가 바뀌면 입력 주소가 맞는지 먼저 확인합니다. `FileNotFoundError`는 파일 이름·주소·압축 해제 상태를 확인하는 단서이고 `KeyError`는 실제 헤더와 찾는 이름표를 비교하는 단서입니다. 파일은 찾았지만 "뜨거움"을 숫자로 읽으려 하면 `ValueError` 단계가 됩니다. 세 오류가 같은 화면에 보여도 해결할 경계가 다릅니다. 저장 후 에는 결과 파일을 다시 열어 `count`와 `average`가 기대값인지 확인합니다. 이전 실행 파일이 남아 있을 수 있으므로 입력명 과 실행 시각도 기록합니다.

새 상황에 독립적으로 적용하기

복습·도전: 입력 헤더는 `device_id,value`이고 한 행은 `D09,0`입니다. 프로그램은 `temperature_c`를 찾다가 실패했습니다. 파일 열기·헤더 찾기·숫자 변환 중 어느 단계인지 판단하고 원문을 보존한 수정 계획과 올바른 JSON 수치 표현을 쓰세요.

정답과 오답의 이유

헤더 찾기 단계의 문제입니다. 실행 계약이 value를 쓰도록 정해졌다면 접근 키를 value로 맞추되 계약과 원문 헤더를 먼저 확인합니다. 수치 0은 실제 값이므로 JSON에서는 0이며 null이 아닙니다. 주소 수정이나 0 삭제는 현재 증거에 맞는 해결이 아닙니다.

핵심 세 질문과 해설

- 1. **CSV 값은 숫자로 자동 읽히나요?** 기본 읽기는 문자열이므로 의미에 맞는 해석 단계를 둡니다.
- 2. **출력했으면 저장도 끝났나요?** 화면 표시와 파일 쓰기·재읽기 확인은 다른 작업입니다.
- 3. **오류 종류를 왜 나누나요?** 파일·구조·값의 어느 경계에서 실패했는지 좁혀 올바른 부분을 고치기 위해서입니다.

개념. 경로는 파일의 저장 주소입니다. 현재 터미널 폴더와 코드 폴더는 다를 수 있습니다. Path(**file**)로 코드 위치를 찾으면 그 위치를 기준으로 자료 주소를 만들 수 있습니다.

왜 필요한가. 터미널을 다른 곳에서 열어도 같은 자료를 읽도록 준비합니다. 긴 준비 줄을 외우기보다 code·work·data의 상대 배치를 이해하는 것이 목적입니다.

함께 보는 예. course/code 또는 course/work의 코드에서 parent.parent는 course입니다. 그 아래 data/notice.txt를 읽습니다. UTF-8은 한글을 글자로 해석하는 인코딩 약속입니다.

헛갈리는 경우. FileNotFoundError는 주소와 이름의 문제를 먼저 확인해야 합니다. 아직 파일을 읽지 못했는데 숫자 변환을 수정해도 원인을 해결하지 못합니다.

전체 코드

```
from pathlib import Path
base = Path(__file__).resolve().parent.parent
source = base / "data" / "notice.txt"
text = source.read_text(encoding="utf-8")
print(text.strip())
```

각 줄의 뜻

줄	코드	뜻
1	from pathlib import Path	파일 주소 도구를 가져옵니다.
2	base = Path(__file__).resolve().parent.parent	코드의 두 상위 폴더 course를 찾습니다.
3	source = base / "data" / "notice.txt"	입력 파일 주소를 만듭니다.
4	text = source.read_text(encoding="utf-8")	UTF-8 텍스트를 읽습니다.
5	print(text.strip())	앞뒤 공백과 줄바꿈을 정리해 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A07_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A07_example.py
```

원본 코드의 예상 출력:

```
센서 점검 예제입니다.
```

A07 · 따라 하기와 작성칸

A07 · 파일 경로와 텍스트 읽기

추가 원문·작업 파일:

- [A07_input.md](#)
- [A07_example.py](#)
- [notice.txt](#)
- [A07_notes.md](#)
- [A07_example.py](#)

목적: 터미널을 다른 곳에서 열어도 같은 자료를 읽도록 준비합니다. 긴 준비 줄을 외우기보다 code·work·data의 상대 배치를 이해하는 것이 목적입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A07_input.md](#)
- [A07_example.py](#)
- [A07_notes.md](#)
- [A07_example.py](#)

단계

1. code·work·data 배치를 그립니다.
2. 실제 파일명을 확인합니다.
3. 같은 코드가 다른 현재 위치에서도 읽는지 비교합니다.

4. 없는 파일의 오류와 내용 오류를 구별합니다.

예상 결과: 안내 한 줄 출력. 없는 파일명은 FileNotFoundError.

확인 방법: data만 다른 곳으로 옮겨도 자동으로 찾나요?

오류 해결: 압축 안에서 열었는지, 확장자가 두 번 붙었는지, course의 배치가 유지됐는지 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A08 · CSV를 이름표가 있는 행으로 읽기

개념. csv.DictReader는 헤더를 키로 삼아 각 데이터 행을 딕셔너리로 읽습니다. 기본 셀값은 문자열이므로 온도 계산 전에 의미에 맞는 해석이 필요합니다.

왜 필요한가. 열 번호를 외우기보다 temperature_c라는 이름으로 읽으면 처리할 항목이 드러납니다. 실제 헤더가 약속과 맞아야 같은 이름으로 읽을 수 있습니다.

함께 보는 예. 입력은 S01의 22.5, S02의 25.0, S03의 빈 온도, S04의 0.0 네 건입니다. 첫 온도의 화면 모양은 숫자 같지만 type은 str입니다.

헷갈리는 경우. 헤더를 temperature로 바꾸면 temperature_c라는 키가 저절로 생기지 않습니다. DictReader가 업무상 비슷한 단어를 연결하지 않습니다.

전체 코드

```
import csv

from pathlib import Path

source = Path(__file__).resolve().parent.parent / "data" / "readings.csv"

with source.open(encoding="utf-8", newline="") as file:
    rows = list(csv.DictReader(file))

print(len(rows))

print(rows[0]["sensor_id"])

print(rows[0]["temperature_c"], type(rows[0]["temperature_c"]).__name__)
```

각 줄의 뜻

줄	코드	뜻
1	<code>import csv</code>	CSV 표준 도구를 가져옵니다.
2	<code>from pathlib import Path</code>	파일 주소 도구를 가져옵니다.
3	<code>source = Path(__file__).resolve().parent.parent / "data" / "readings.csv"</code>	기초 CSV 주소를 만듭니다.
4	<code>with source.open(encoding="utf-8", newline="") as file:</code>	인코딩과 CSV 줄바꿈 설정으로 엽니다.
5	<code>rows = list(csv.DictReader(file))</code>	각 행을 헤더 이름의 딕셔너리로 읽습니다.
6	<code>print(len(rows))</code>	데이터 건수를 출력합니다.
7	<code>print(rows[0]["sensor_id"])</code>	첫 기록의 센서명을 읽습니다.
8	<code>print(rows[0]["temperature_c"], type(rows[0]["temperature_c"]).__name__)</code>	첫 온도와 실제 자료형을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A08_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A08_example.py
```

원본 코드의 예상 출력:

```
4
S01
22.5 str
```

A08 · 따라 하기와 작성칸

A08 · CSV를 이름표가 있는 행으로 읽기

추가 원문·작업 파일:

- [A08_input.md](#)

- [A08_example.py](#)

- [readings.csv](#)
- [A08_notes.md](#)
- [A08_example.py](#)

목적: 열 번호를 외우기보다 temperature_c라는 이름으로 읽으면 처리할 항목이 드러납니다. 실제 헤더가 약속과 맞아야 같은 이름으로 읽을 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A08_input.md](#)
- [A08_example.py](#)
- [A08_notes.md](#)
- [A08_example.py](#)

단계

1. 헤더와 키의 대응을 표시합니다.
2. 헤더를 제외한 네 건을 셉니다.
3. 온도의 값 모양과 자료형을 나눠 적습니다.
4. 빈 온도가 어떤 문자열인지 예상합니다.

예상 결과: 4 / S01 / 22.5 str. 헤더는 데이터가 아닙니다.

확인 방법: 빈 셀은 자동으로 None이 되나요?

오류 해결: KeyError는 실제 헤더와 키를 비교합니다. 공백·인코딩도 실제 원문을 확인한 뒤 판단합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A09 · JSON 결과를 새 파일로 저장하기

개념. json.dumps는 프로그램 값을 JSON 텍스트로 표현합니다. 그 텍스트를 경로에 써야 결과 파일이 생깁니다. 입력 읽기와 결과 저장은 별도 작업입니다.

왜 필요한가. 화면 출력만 있으면 다음 사람이 결과를 재사용하기 어렵습니다. 입력 원자료를 덮어쓰지 않도록 work의 결과 경로를 명확히 정합니다.

함께 보는 예. 건수 4와 미측정 1을 JSON으로 저장합니다. ensure_ascii=False는 한글을 읽기 쉬운 모양으로 남기고 indent=2는 들여쓰기를 넣습니다. 저장한 파일을 다시 읽어 값이 같은지 봅니다.

헛갈리는 경우, .json이라는 이름만으로 내용이 JSON이 되지는 않습니다. 디렉터리 화면 문자열을 그대로 쓰기보다 JSON 변환 도구를 사용합니다.

전체 코드

```
import json

from pathlib import Path

base = Path(__file__).resolve().parent.parent

summary = {"건수": 4, "미측정": 1}

text = json.dumps(summary, ensure_ascii=False, indent=2)

target = base / "work" / "output_summary.json"

target.write_text(text, encoding="utf-8")

loaded = json.loads(target.read_text(encoding="utf-8"))

print(loaded["건수"], loaded["미측정"])
```

각 줄의 뜻

줄	코드	뜻
1	<code>import json</code>	JSON 도구를 가져옵니다.
2	<code>from pathlib import Path</code>	주소 도구를 가져옵니다.
3	<code>base = Path(__file__).resolve().parent.parent</code>	course를 찾습니다.
4	<code>summary = {"건수": 4, "미측정": 1}</code>	결과 객체를 만듭니다.
5	<code>text = json.dumps(summary, ensure_ascii=False, indent=2)</code>	JSON 텍스트로 표현합니다.
6	<code>target = base / "work" / "output_summary.json"</code>	work의 결과 주소를 정합니다.
7	<code>target.write_text(text, encoding="utf-8")</code>	UTF-8로 저장합니다.
8	<code>loaded = json.loads(target.read_text(encoding="utf-8"))</code>	저장한 파일을 다시 읽어 해석합니다.
9	<code>print(loaded["건수"], loaded["미측정"])</code>	다시 읽은 두 값을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A09_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A09_example.py
```

원본 코드의 예상 출력:

```
4 1
```

작은 수정: "건수": 4를 "건수": 5로 바꿉니다.

수정 후 예상 출력:

```
5 1
```

A09 · 따라 하기와 작성칸

A09 · JSON 결과를 새 파일로 저장하기

추가 원문·작업 파일:

- [A09_input.md](#)
- [A09_example.py](#)
- [A09_notes.md](#)
- [A09_example.py](#)
- [output_summary.json](#)

목적: 화면 출력만 있으면 다음 사람이 결과를 재사용하기 어렵습니다. 입력 원자료를 덮어쓰지 않도록 work의 결과 경로를 명확히 정합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A09_input.md](#)
- [A09_example.py](#)
- [A09_notes.md](#)
- [A09_example.py](#)

단계

1. data와 work의 역할을 구별합니다.
2. 실행 후 output_summary.json을 엽니다.
3. 건수만 5로 바꾸고 다시 실행합니다.

4. 파일 저장 줄과 화면 출력 줄을 표시합니다.

예상 결과: 처음 4 1, 수정 후 5 1. 저장 파일의 숫자도 바뀝니다.

확인 방법: print와 write_text는 같은 행동인가요?

오류 해결: 쓰기 권한이 없으면 강사와 저장 위치를 확인합니다. 원자료를 출력 주소로 바꾸지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

4교시 · 오류를 단서로 읽기

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 오류를 단서로 확인하기

먼저 할 활동: A10, A11, A12

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 미측정과 잘못된 숫자의 구별

사례로 이해하기 · 미측정·잘못된 글자·실제 0의 세 갈래

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

수업용 측정 파일에 빈 문자열, "0", "추움" 세 원문이 있습니다. 검사를 통과시키려고 모두 0으로 바꾸면 세 기록이 실제 0°C로 합쳐집니다. 빈 문자열은 이번 데이터 사전에서 미측정이고, "0"은 측정한 숫자 표현이며, "추움"은 숫자 칸에 온 다른 종류의 내용입니다. 먼저 원문을 보존하고 어떤 갈래에 속하는지 정합니다. 오류 처리는 숨기는 행동이 아니라 분류와 후속 확인을 기록하는 행동입니다.

해석 절차의 첫 질문은 원문이 빈 문자열과 정확히 같은지입니다. 같으면 None으로 반환해 미측정을 보존합니다. 아니면 숫자로 변환합니다. "0"은 0.0이 되지만 "추움"은 ValueError가 납니다. 이 예에서는 ValueError를 잡아 장치번호와 원문을 이슈로 남깁니다. 처리할 수 없는 숫자 글자 오류를 잡는 것과 주소 오류까지 무조건 숨기는 것은 다릅니다. except ValueError는 해당 변환·규칙 오류만 말고 파일이 없다는 오류의 해결은 별도입니다.

완성 분류표는 R01/빈 문자열/미측정, R02/"0"/수치 0.0, R03/"추움"/이슈입니다. 수치 평균을 계산한다면 R02만 대상으로 하므로 평균은 0.0입니다. R01을 0으로 채워 평균에 넣으면 대상 수가 늘고 미측정 사실이 사라집니다. R03을 None으로 바꾸면 잘못 적힌 내용이 있었다는 근거가 사라집니다. 오류 원문을 보존한 후 담당자에게 어떤 입력을 의도했는지 묻습니다. 공백 한 칸 " "도 빈 문자열과 그대로는 다르며 이 수업 계약에는 자동 공백 정리가 없습니다. 새 정리 규칙이 필요하다면 먼저 원문과 변경 이유를 기록하고 별도 약속으로 정해야 합니다.

새 상황에 독립적으로 적용하기

복습·도전: 수량 칸 원문이 빈 문자열, "0", "여섯", " "입니다. 계약은 정확한 빈 문자열만 미확인, 숫자만 계산 대상으로 정합니다. 네 갈래의 분류와 담당자에게 남길 이슈 원문을 작성하세요.

정답과 오답의 이유

빈 문자열은 미확인, "0"은 수치 0, "여섯"과 공백 한 칸은 현재 계약의 숫자 해석 이슈입니다. 두 이슈 원문을 그대로 남깁니다. 공백을 자동 삭제해 미확인으로 합치는 답은 명시되지 않은 정리 규칙을 추가한 것입니다.

핵심 세 질문과 해설

1. **미측정도 오류인가요?** 계약에서 허용한 값 없음과 잘못된 입력을 구별합니다.
2. **예외를 잡으면 실제 값을 알게 되나요?** 아닙니다. 실패 원문을 보존하고 추가 확인할 근거를 얻습니다.
3. **0을 평균에서 빼나요?** 실제 수치 0은 포함하고 값 없음만 제외합니다.

개념. 빈값 규칙과 숫자 변환은 다른 단계입니다. 이 자료의 빈 온도는 미측정이므로 None으로 보존하고 숫자가 아닌 글자는 잘못된 입력으로 알립니다.

왜 필요한가. 모든 변환 실패를 None으로 바꾸면 누락과 잘못된 기록을 구별하지 못합니다. 원문과 실패 이유를 남겨야 확인 요청을 할 수 있습니다.

함께 보는 예. 빈 문자열은 None, "0.0"은 숫자 0.0, "뜨거움"은 ValueError입니다. 먼저 빈 문자열인지 확인하고 나머지에 숫자 변환을 적용합니다.

헛갈리는 경우. 빈 문자열을 0으로 대신 채우면 미측정을 실제 0°C로 바꿉니다. 계산을 편하게 하려고 확인되지 않은 값을 보충하지 않습니다.

전체 코드

```
def parse_temperature(raw):
    if raw == "":
        return None
    return float(raw)

print(parse_temperature("22.5"))
print(parse_temperature(""))
print(parse_temperature("0.0"))
```

각 줄의 뜻

줄	코드	뜻
1	<code>def parse_temperature(raw):</code>	원문 온도를 해석하는 함수를 정의합니다.
2	<code>if raw == "":</code>	빈 문자열 여부를 확인합니다.
3	<code>return None</code>	계약에 따라 None을 반환합니다.
4	<code>return float(raw)</code>	나머지를 숫자로 변환합니다.
5	<code>print(parse_temperature("22.5"))</code>	일반 숫자를 확인합니다.
6	<code>print(parse_temperature(""))</code>	미측정을 확인합니다.
7	<code>print(parse_temperature("0.0"))</code>	실제 0을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A10_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A10_example.py
```

원본 코드의 예상 출력:

```
22.5
None
0.0
```

A10 · 따라 하기와 작성칸

A10 · 미측정과 잘못된 숫자의 구별

목적: 모든 변환 실패를 None으로 바꾸면 누락과 잘못된 기록을 구별하지 못합니다. 원문과 실패 이유를 남겨야 확인 요청을 할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A10_input.md](#)

- [A10_example.py](#)

- [A10_notes.md](#)
- [A10_example.py](#)

단계

1. 원문·반환값·업무 의미를 표로 적습니다.
2. 뜨거움과 빈칸의 차이를 설명합니다.
3. 오류 원문과 종류를 기록합니다.
4. 실제값이 미확인일 때 남길 질문을 씁니다.

예상 결과: 22.5/None/0.0을 구별하며 글자는 ValueError.

확인 방법: 잘못된 숫자를 미측정으로 바꾸면 무엇을 잃나요?

오류 해결: None과 문자열 "None"은 다릅니다. 후자는 숫자로 읽으면 실패합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A11 · 오류의 위치·종류·원값

개념. 오류 기록은 실패한 파일과 줄, 오류 종류, 관련 값의 단서를 보여 줍니다. traceback은 호출 경로를 나타내며 마지막 줄에서 오류 종류와 설명을 찾습니다.

왜 필요한가. 파일 없음·키 없음·숫자 변환 실패는 다음 행동이 다릅니다. 이름만 암기하기보다 실패한 작업과 입력을 연결합니다.

함께 보는 예. FileNotFoundError는 주소, KeyError는 헤더와 키, ValueError는 숫자로 해석하려던 원문을 확인합니다. 근거를 보기 전에 전체 코드를 교체하지 않습니다.

헛갈리는 경우. float(raw)에서 실패했다고 float 도구 자체가 잘못된 것은 아닐 수 있습니다. 숫자 자리에 '뜨거움'이 들어온 입력 문제를 확인해야 합니다.

제공 입력

```
카드1: source.read_text() / FileNotFoundError / missing.csv
카드2: row['temperature_c'] / KeyError / temperature_c
카드3: float('뜨거움') / ValueError / could not convert string to float
```

A11 · 따라 하기와 작성칸

A11 · 오류의 위치·종류·원값

목적: 파일 없음·키 없음·숫자 변환 실패는 다음 행동이 다릅니다. 이름만 암기하기보다 실패한 작업과 입력을 연결합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A11_input.md](#)

- [A11_notes.md](#)

단계

1. 실패 작업·종류·경로 또는 값을 표시합니다.
2. 사실과 추정 원인을 나눕니다.
3. 다음 확인 동작을 하나씩 정합니다.
4. 오류 단서를 보존한 전달 기록을 씁니다.

예상 결과: 주소·헤더·원값 확인을 구별합니다.

확인 방법: ValueError면 모두 같은 값으로 고쳐도 되나요?

오류 해결: 명령·관련 줄·마지막 오류를 남깁니다. 길다는 이유로 단서를 모두 지우지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A12 · try와 except로 예상 오류 기록

개념. try에는 실패할 수 있는 작업을 쓰고 except에는 특정 예외가 났을 때 할 일을 씁니다. 오류를 처리하는 일은 문제를 없던 일로 만드는 것이 아니라 기록과 다음 행동을 정하는 일입니다.

왜 필요한가. 한 값이 잘못되어도 원문과 이유를 남길 수 있습니다. 다룰 오류 종류를 좁히면 예상하지 못한 문제를 숨기지 않습니다.

함께 보는 예. '뜨거움'을 float로 읽으면 ValueError가 납니다. 그때 원문과 확인 필요 메시지를 출력합니다. 정상값 24.0을 주면 숫자를 출력합니다.

헛갈리는 경우. except: pass로 모든 오류를 무시하면 어떤 기록이 사라졌는지 모릅니다. 이 코드는 ValueError만 다루며 모든 오류를 복구한다고 설명하지 않습니다.

전체 코드

```
raw = "뜨거움"

try:
    value = float(raw)
    print(value)
except ValueError:
    print("숫자 확인 필요:", raw)
```

각 줄의 뜻

줄	코드	뜻
1	<code>raw = "뜨거움"</code>	수치로 읽을 원문입니다.
2	<code>try:</code>	실패 가능 작업을 시작합니다.
3	<code>value = float(raw)</code>	숫자로 해석합니다.
4	<code>print(value)</code>	성공한 숫자를 출력합니다.
5	<code>except ValueError:</code>	변환 오류의 처리 묶음입니다.
6	<code>print("숫자 확인 필요:", raw)</code>	문제 원문을 포함해 메시지를 남깁니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A12_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A12_example.py
```

원본 코드의 예상 출력:

```
숫자 확인 필요: 뜨거움
```

작은 수정: "뜨거움"를 "24.0"로 바꿉니다.

수정 후 예상 출력:

```
24.0
```

A12 · 따라 하기와 작성칸

A12 · try와 except로 예상 오류 기록

목적: 한 값이 잘못되어도 원문과 이유를 남길 수 있습니다. 다룰 오류 종류를 좁히면 예상하지 못한 문제를 숨기지 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A12_input.md](#)

- [A12_example.py](#)

- [A12_notes.md](#)
- [A12_example.py](#)

단계

1. 성공과 오류의 실행 줄을 표시합니다.
2. raw를 24.0으로 바꿔 비교합니다.
3. 오류 기록과 실제값 확정을 구별합니다.
4. 원문을 메시지에 남긴 이유를 설명합니다.

예상 결과: 오류는 확인 필요, 정상값은 24.0 출력.

확인 방법: except를 실행하면 실제 온도를 알아낸 것인가요?

오류 해결: 오류를 숨기려고 except의 범위를 무조건 넓히지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

5교시 · 검사 규칙과 재확인

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 허용 범위와 수정 근거

먼저 할 활동: A13, A14, A15

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A13 · 범위 규칙을 함수에서 확인하기

사례로 이해하기 · 허용 범위의 바깥과 경계를 검사하기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

접수 시스템은 물품 수량을 0개부터 20개까지 받습니다. 숫자로 바꿀 수 있는 21도 업무 계약에는 맞지 않습니다. 숫자 변환과 허용 범위 검사는 다른 문턱입니다. 먼저 “스물”은 숫자 표현이 아니라 변환에서 멈추고, 21은 숫자로 읽힌 뒤 범위에서 멈춥니다. 0은 실제 수량 없음이고 이번 계약에 포함됩니다. 같은 ValueError라는 결과가 나올 수 있어도 어떤 규칙과 원문 때문에 실패했는지 기록합니다.

$0 \leq \text{수량} \leq 20$ 을 수직선으로 그리면 두 끝점이 포함됩니다. 바깥은 '0보다 작음 또는 20보다 큼'입니다. or는 둘 중 하나라도 참이면 참이고 and는 둘 다 참이어야 참입니다. 완성 검수표는 -1 거부, 0 허용, 10 허용, 20 허용, 21 거부입니다. 경계 0과

20을 보지 않으면 <와 <=를 바꾼 실수를 놓칠 수 있습니다. '0보다 작고 동시에 20보다 큼'이라는 and 조건은 한 숫자에 성립하지 않아 범위 밖을 놓칩니다.

실제 상품의 재고 한도인지 주문 수량 한도인지도 계약에 적어야 합니다. 같은 숫자 20을 모든 업무의 기준으로 재사용하지 않습니다. 21을 자동으로 20으로 잘라 저장하면 검사 대신 원자로 변경을 수행한 것입니다. 담당자의 확인 없이 실제 수량을 고치지 않고 거부 사유와 원문을 남깁니다. 수정 검증은 실패 입력 한 건만 다시 보는 것이 아닙니다. 기준 비교를 바꿨다면 정상·양쪽 경계·양쪽 바깥의 다섯 사례를 다시 봅니다. 통과한 사례 수와 검사하지 않은 종류를 구별해서 보고합니다. 이 계약은 정수 개수를 받으며 1.5개를 허용할지는 별도 결정해야 합니다. 범위 통과만으로 정수 조건이나 실물 재고의 정확성이 증명되지는 않습니다.

새 상황에 독립적으로 적용하기

복습·도전: 수업용 배터리 기록의 허용 충전율은 0~100%입니다. -0.1, 0, 50, 100, 100.1의 기대 결과와 바깥을 찾는 논리 문장을 쓰세요. 100.1을 100으로 자동 변경해도 되는지 근거를 설명하세요.

정답과 오답의 이유

양 끝 -0.1·100.1은 거부, 0·50·100은 허용입니다. '0 미만 또는 100 초과'가 바깥입니다. 자동 변경은 원문을 바꾸므로 현재 검사 계약만으로는 허용되지 않습니다. and로 쓰면 두 바깥 조건을 동시에 만족하는 값이 없어 오류를 놓칩니다.

핵심 세 질문과 해설

1. **변환 성공이 허용 성공인가요?** 숫자 표현과 업무 범위·정수 조건은 별도 검사입니다.
2. **범위 밖에 왜 or를 쓰나요?** 아래쪽이나 위쪽 중 하나만 벗어나도 거부해야 하기 때문입니다.
3. **통과가 실제 정확성을 뜻하나요?** 입력 규칙에 맞았다는 뜻이며 실제 측정·기록 정확성은 따로 확인합니다.

개념. 숫자로 변환할 수 있다는 것과 허용 범위 안에 있다는 것은 다른 검사입니다. 값이 0~50인지 확인하고 벗어나면 이유를 예외로 알립니다.

왜 필요한가. 51.0은 숫자이므로 float는 성공하지만 실습 입력 규칙에는 맞지 않습니다. 변환과 규칙 검사를 분리하여 읽어야 합니다.

함께 보는 예. 23.5는 반환하고 51.0은 범위 확인 필요를 알립니다. 0과 50은 허용 경계입니다. 이 범위는 실습의 입력 약속이며 실제 장비 안전 기준이 아닙니다.

헛갈리는 경우. 51.0을 자동으로 50으로 자르면 원자료를 바꿉니다. 실제값을 모르면 원문을 보존하고 확인 요청을 남깁니다.

전체 코드

```
def checked_temperature(raw):
    value = float(raw)
    if value < 0 or value > 50:
        raise ValueError("범위 확인 필요")
    return value
print(checked_temperature("23.5"))
```

각 줄의 뜻

줄	코드	뜻
1	<code>def checked_temperature(raw):</code>	수치 원문을 검사하는 함수를 정의합니다.
2	<code>value = float(raw)</code>	숫자로 해석합니다.
3	<code>if value < 0 or value > 50:</code>	아래 또는 위 경계를 벗어나는지 묻습니다.
4	<code>raise ValueError("범위 확인 필요")</code>	범위 문제를 예외로 알립니다.
5	<code>return value</code>	허용 숫자를 반환합니다.
6	<code>print(checked_temperature("23.5"))</code>	정상 입력으로 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A13_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A13_example.py
```

원본 코드의 예상 출력:

```
23.5
```

작은 수정: "23.5"를 "50"로 바꿉니다.

수정 후 예상 출력:

```
50.0
```

A13 · 따라 하기와 작성칸

A13 · 범위 규칙을 함수에서 확인하기

목적: 51.0은 숫자이므로 float는 성공하지만 실습 입력 규칙에는 맞지 않습니다. 변환과 규칙 검사를 분리하여 읽어야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A13_input.md](#)

- [A13_example.py](#)

- [A13_notes.md](#)
- [A13_example.py](#)

단계

1. 변환과 범위 검사를 표시합니다.
2. 0·50·51.0의 기대 결과를 씁니다.
3. 경계와 오류 입력을 비교합니다.
4. 원문과 확인 이유를 남깁니다.

예상 결과: 0·50은 허용하고 51.0은 범위 오류입니다.

확인 방법: 범위 통과가 실제 센서 정확도를 증명하나요?

오류 해결: or를 and로 바꿨다면 두 조건의 뜻을 읽습니다. 동시에 0보다 작고 50보다 큰 값은 찾을 수 없습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A14 · 한 값 수정 뒤 같은 사례 재확인

개념. 수정 검증은 확인된 근거로 한 부분을 바꾸고 실패 사례와 기존 성공 사례를 다시 확인하는 일입니다. 오류가 사라졌는지뿐 아니라 원래 의미가 유지되는지 봅니다.

왜 필요한가. 한 문제를 고치며 미측정을 0으로 바꾸거나 경계를 바꿀 수 있습니다. 원문·지시·수정 위치·재확인을 함께 남겨야 이유가 전달됩니다.

함께 보는 예. 지시서에서 S05의 ‘뜨거움’은 23.5로 확정되었고 S06의 51.0은 미확인입니다. 전자는 근거로 수정하고 후자는 원문 이슈로 남깁니다.

헛갈리는 경우. 문제가 두 건이라고 둘 다null로 바꾸면 잘못된 숫자와 미측정을 합칩니다. 확인된 수정과 미확인 보류를 구별합니다.

제공 입력

확정 지시: S05 10:05는23.5°C로 확인됨.
 미확정: S06 10:05의51.0은 실제값미확인.
 재확인: 정상22.5 / 빈칸 / 실제0.0 / 경계50.

A14 · 따라 하기와 작성칸

A14 · 한 값 수정 뒤 같은 사례 재확인

추가 원문·작업 파일:

- [A14_input.md](#)
- [bad_readings.csv](#)
- [A14_notes.md](#)
- [bad_readings_work.csv](#)
- [correction_notes.md](#)

목적: 한 문제를 고치며 미측정을 0으로 바꾸거나 경계를 바꿀 수 있습니다. 원문·지시·수정 위치·재확인을 함께 남겨야 이유가 전달됩니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A14_input.md](#)
- [A14_notes.md](#)

단계

1. 확정과 미확정 지시를 나눕니다.
2. work/correction_notes.md에 전후값과 근거를 씁니다.
3. work/bad_readings_work.csv에서 S05 한 값만 고칩니다.
4. 정상·미측정·0·범위의 재확인 표를 작성합니다.

예상 결과: S05만 23.5로 수정하고 S06원문과 확인 이유를 남깁니다.

확인 방법: 이슈 건수만 줄면 좋은 수정인가요?

오류 해결: 확정되지 않은 실제값은 추측하지 않습니다. 어느 값이 왜 미확인인지 씁니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A15 · 입력·처리·출력의 실행 계약

개념. 실행 계약은 받을 입력 파일과 결과 파일의 위치·뜻을 정한 약속입니다. 함수의 입력·반환을 프로그램 전체의 파일 수준으로 확장합니다.

왜 필요한가. 원본과 수정 사본 중 무엇을 읽었는지 모르면 결과 대조가 틀릴 수 있습니다. 실행 완료와 데이터 이슈가 없다는 사실도 별도입니다.

함께 보는 예. 검사기는 data/bad_readings.csv를 읽고 work/check_summary.json과 check_issues.json을 만듭니다. 6건 중 숫자·미측정 4건을 받아들이고 2건을 이슈로 남깁니다.

헛갈리는 경우. 정상 종료해도 이슈는 있을 수 있습니다. 여기서 종료 코드 0은 실행 완료를 뜻하며 입력 전체가 정확하다는 증명은 아닙니다.

제공 입력

입력6건 → 수락4건+이슈2건.
 수락4건 → 수치3건+미측정1건.
 미측정은 평균에서 제외, 실제0은 포함.

A15 · 따라 하기와 작성칸

A15 · 입력·처리·출력의 실행 계약

추가 원문·작업 파일:

- [A15_input.md](#)
- [bad_readings.csv](#)
- [A15_notes.md](#)

목적: 원본과 수정 사본 중 무엇을 읽었는지 모르면 결과 대조가 틀릴 수 있습니다. 실행 완료와 데이터 이슈가 없다는 사실도 별도입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A15_input.md](#)
- [A15_notes.md](#)

단계

1. 세 파일의 위치를 그림니다.
2. 미측정과 실제 0의 집계 차이를 적습니다.
3. 실행 완료와 이슈 0건을 구별합니다.
4. 다음 활동의 예상 건수를 먼저 기록합니다.

예상 결과: 전체 6,수락 4,이슈 2,미측정 1,수치 3.

확인 방법: 미측정도 분모에 넣어 4로 나누면 무엇이 바뀌나요?

오류 해결: 전체=수락+이슈, 수락=수치+미측정의 두 관계를 나누어 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

6교시 · 로그 검사기와 사용 안내

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 로그 검사기의 전체 흐름

먼저 할 활동: A16, A17, A18

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 로그 검사기 전체 흐름

사례로 이해하기 · 분류 건수와 평균 분모를 보존하기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

로그 검사기를 넘겨받은 사람이 요약의 accepted를 실제 숫자 수라고 읽었습니다. 그러나 기존 검사기에서 accepted는 수치와 미측정을 합한 수락 건수입니다. 이름만 보고 의미를 추측하지 말고 결과 계약과 계산 줄을 확인해야 합니다. 합성 원문 다섯 건은 "10", 빈 문자열, "20", "오류", "0"입니다. 수치는 세 건, 미측정은 한 건, 이슈는 한 건입니다. 입력 전체와 분류된 부분이 빠짐없이 맞는지 검산합니다.

이 프로그램의 관계는 $input=accepted+issues$, $accepted=numeric+missing$ 입니다. 다섯 건 사례에서는 $5=4+1$, $4=3+1$ 입니다. 평균은 숫자 세 건의 합 $10+20+0=30$ 을 숫자 건수 3으로 나누어 10입니다. 수락 건수 4로 나누면 7.5가 되어 미측정을 0처럼 처리합니다. 이슈 건수까지 포함해 5로 나누면 6이 됩니다. 같은 원문에서 분모만 바뀌도 업무 의미가 바뀌므로 평균 대상과 건수를 함께 보고합니다.

한 행의 추적은 원문→해석→분류→누적 순서입니다. 잘못된 글자가 나오면 이슈 목록에 원문을 남기고 continue로 다음 행으로 넘어갑니다. 문제 행이 수치 목록에도 들어가면 중복 분류가 되어 두 검산 관계가 깨질 수 있습니다. 모든 수치가 없을 때 평균은 None입니다. 0이라고 쓰면 실제 평균 0을 계산했다는 오해가 생깁니다. 헤더만 있는 입력은 전체 0건이고 모든 값이 빈 입력은 전체 기록은 있으나 수치가 0건입니다. 둘 다 평균이 없지만 같은 자료는 아닙니다. 결과 안내에는 입력 파일명·명령·실제 출력·미확인 범위·변경 이유를 남깁니다. 현재 검사는 합성 온도값 규칙을 확인하며 모든 필드나 센서의 실제 성능을 보증하지 않습니다.

새 상황에 독립적으로 적용하기

복습·도전: 원문은 "5", "15", 빈 문자열, "뜨거움", 빈 문자열, "0"입니다. 전체·수치·미측정·이슈·수락 건수와 평균을 구하고 두 검산식을 쓰세요. 수치가 전혀 없는 경우의 평균 표현도 적으세요.

정답과 오답의 이유

전체 6, 수치 3, 미측정 2, 이슈 1, 수락 5입니다. $6=5+1$, $5=3+2$ 이고 평균은 $20\div3$ 을 소수 둘째 자리로 반올림한 6.67입니다. 수치가 없으면 None입니다. 수락 5를 분모로 쓰면 4가 되어 미측정을 포함한 잘못된 평균입니다.

핵심 세 질문과 해설

1. 수락 건수는 수치 건수인가요? 이 검사기에서는 수치와 미측정을 합하므로 결과 계약을 읽습니다.

2. **continue가 필요한 이유는?** 이슈로 분류한 행을 정상 처리로 보내지 않고 다음 행으로 넘어갑니다.

3. **평균 None과 0은 같은가요?** None은 계산할 수치가 없음이며 0은 실제 수치 평균이 0이라는 값입니다.

개념. 검사기는 한 행씩 온도를 해석하여 정상 수치·미측정·이슈를 나눕니다. 함수는 한 값의 규칙을 담당하고 반복문은 여러 행에 같은 규칙을 적용합니다.

왜 필요한가. 함수·파일·예외를 하나의 업무에 연결합니다. 긴 코드는 준비·정의·행 처리·집계·저장으로 나눠 추적합니다. 전체를 암기하거나 빈 편집기에서 다시 작성하는 것이 목표는 아닙니다.

함께 보는 예. 실제 수치는 22.5·25.0·0.0으로 합 47.5, 개수 3, 평균 15.83입니다. 빈 온도 1건은 미측정, 뜨거움과 51.0은 원문 이슈로 남깁니다. 0도 실제 측정이므로 평균에 포함합니다.

헛갈리는 경우. 평균이 낮아 보인다는 이유로 0을 빼지 않습니다. 원문과 계약을 봅니다. 이 프로그램은 온도 규칙에 집중하며 모든 필드와 실제 측정의 진위를 검사하지 않습니다.

전체 코드

```
import csv
import json
from pathlib import Path

def parse_temperature(raw):
    if raw == "":
        return None
    value = float(raw)
    if value < 0 or value > 50:
        raise ValueError("범위 확인 필요")
    return value

base = Path(__file__).resolve().parent.parent
source = base / "data" / "bad_readings.csv"
with source.open(encoding="utf-8", newline="") as file:
    rows = list(csv.DictReader(file))
numbers = []
issues = []
missing = 0
for row in rows:
    try:
        value = parse_temperature(row["temperature_c"])
    except ValueError:
        issues.append({"sensor_id": row["sensor_id"], "raw": row["temperature_c"]})
        continue
    if value is None:
        missing = missing + 1
    else:
        numbers.append(value)
average = None
if len(numbers) > 0:
    average = round(sum(numbers) / len(numbers), 2)
summary = {"input": len(rows), "accepted": len(numbers) + missing,
           "missing": missing, "issues": len(issues), "average": average}
(base / "work" / "check_summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
(base / "work" / "check_issues.json").write_text(
```

```
    json.dumps(issues, ensure_ascii=False, indent=2), encoding="utf-8")
print(summary["input"], summary["accepted"], summary["missing"], summary["issues"])
print(summary["average"])
```

각 줄의 뜻

줄	코드	뜻
1	<code>import csv</code>	CSV 도구를 가져옵니다.
2	<code>import json</code>	JSON 도구를 가져옵니다.
3	<code>from pathlib import Path</code>	경로 도구를 가져옵니다.
4	<code>``</code>	정의 부분을 나누는 빈 줄입니다.
5	<code>def parse_temperature(raw):</code>	온도 해석 함수를 정의합니다.
6	<code>if raw == "":</code>	빈칸인지 확인합니다.
7	<code>return None</code>	미측정을 반환합니다.
8	<code>value = float(raw)</code>	숫자로 변환합니다.
9	<code>if value < 0 or value > 50:</code>	범위를 확인합니다.
10	<code>raise ValueError("범위 확인 필요")</code>	범위 문제를 알립니다.
11	<code>return value</code>	유효 숫자를 반환합니다.
12	<code>``</code>	정의와 실행을 나눕니다.
13	<code>base = Path(__file__).resolve().parent.parent</code>	course 주소를 정합니다.
14	<code>source = base / "data" / "bad_readings.csv"</code>	입력 파일을 선택합니다.
15	<code>with source.open(encoding="utf-8", newline="") as file:</code>	입력 파일을 엽니다.
16	<code>rows = list(csv.DictReader(file))</code>	헤더 키로 행을 읽습니다.
17	<code>numbers = []</code>	숫자 목록을 준비합니다.
18	<code>issues = []</code>	이슈 목록을 준비합니다.
19	<code>missing = 0</code>	미측정 개수를 준비합니다.
20	<code>for row in rows:</code>	한 행씩 처리합니다.
21	<code>try:</code>	해석을 시도합니다.
22	<code>value = parse_temperature(row["temperature_c"])</code>	이번 온도 원문을 함수에 줍니다.

줄	코드	뜻
23	<code>except ValueError:</code>	변환·범위 오류를 처리합니다.
24	<code>issues.append({"sensor_id": row["sensor_id"], "raw": row["temperature_c"]})</code>	센서명과 원문을 이슈로 보존합니다.
25	<code>continue</code>	다음 행으로 넘어갑니다.
26	<code>if value is None:</code>	미측정을 구별합니다.
27	<code>missing = missing + 1</code>	미측정 개수를 늘립니다.
28	<code>else:</code>	숫자인 경우입니다.
29	<code>numbers.append(value)</code>	수치 목록에 넣습니다.
30	<code>average = None</code>	수치가 없을 때의 평균을 준비합니다.
31	<code>if len(numbers) > 0:</code>	숫자가 있을 때만 계산합니다.
32	<code>average = round(sum(numbers) / len(numbers), 2)</code>	실제 수치 개수로 나누어 반올림합니다.
33	<code>summary = {"input": len(rows), "accepted": len(numbers) + missing,</code>	입력·수락 건수로 요약의 시작합니다.
34	<code>"missing": missing, "issues": len(issues), "average": average}</code>	미측정·이슈·평균을 포함합니다.
35	<code>(base / "work" / "check_summary.json").write_text(</code>	요약 파일 쓰기를 시작합니다.
36	<code>json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")</code>	JSON과 UTF-8로 저장합니다.
37	<code>(base / "work" / "check_issues.json").write_text(</code>	이슈 파일 쓰기를 시작합니다.
38	<code>json.dumps(issues, ensure_ascii=False, indent=2), encoding="utf-8")</code>	이슈 원문을 JSON으로 저장합니다.
39	<code>print(summary["input"], summary["accepted"], summary["missing"], summary["issues"])</code>	집계 네 개를 보여 줍니다.
40	<code>print(summary["average"])</code>	평균 또는 None을 보여 줍니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A16_log_checker.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A16_log_checker.py
```

원본 코드의 예상 출력:

```
6 4 1 2
15.83
```

작은 수정: "bad_readings.csv"를 "readings.csv"로 바꿉니다.

수정 후 예상 출력:

```
4 4 1 0
15.83
```

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

A16 · 따라 하기와 작성칸

A16 · 로그 검사기 전체 흐름

추가 원문·작업 파일:

- [A16_input.md](#)
- [A16_log_checker.py](#)
- [bad_readings.csv](#)
- [readings.csv](#)
- [A16_notes.md](#)
- [A16_log_checker.py](#)
- [check_summary.json](#)
- [check_issues.json](#)

목적: 함수·파일·예외를 하나의 업무에 연결합니다. 긴 코드는 준비·정의·행 처리·집계·저장으로 나눠 추적합니다. 전체를 암기하거나 빈 편집기에서 다시 작성하는 것이 목표는 아닙니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A16_input.md](#)

- [A16_log_checker.py](#)

- [A16_notes.md](#)

- [A16_log_checker.py](#)

단계

1. 전체 코드를 역할별 다섯 구간으로 표시합니다.
2. 건수와 평균을 손으로 먼저 확인합니다.
3. 실행 뒤 결과 JSON 두 개를 엽니다.
4. 입력 파일명만 readings.csv로 바꿔 비교합니다.

예상 결과: 오류 포함 6/4/1/2, 정상 4/4/1/0. 평균은 모두 15.83.

확인 방법: continue의 역할은 무엇인가요?

오류 해결: 한 행의 원문→함수→분류를 먼저 추적합니다. 여러 곳을 동시에 바꾸지 않습니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.

- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.

- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 ☐ 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

A17 · 정상·빈 입력·잘못된 입력 비교

개념. 검사기는 서로 다른 입력 집합으로 확인합니다. 헤더만 있는 파일, 전부 미측정인 파일, 숫자 오류가 있는 파일은 다른 경계 상황입니다.

왜 필요한가. 수치가 하나도 없으면 평균 분모가 0입니다. 평균 None은 계산할 수치가 없다는 결과이며 실제 평균 0.0과 의미가 다릅니다.

함께 보는 예. empty_readings.csv는 데이터 0건, missing_readings.csv는 미측정 2건입니다. 둘 다 평균 None이지만 입력수와 미측정수는 다릅니다.

헛갈리는 경우. None이면 모두 실패라고 판정하지 않습니다. 입력 조건에 따라 의도한 결과일 수 있습니다. 어떤 파일로 무엇을 확인했는지 기록합니다.

제공 입력

```
readings.csv4건 / bad_readings.csv6건 / empty_readings.csv헤더만 / missing_readings.csv미측정2건.
work/A16_log_checker.py 안 source 줄의 따옴표 속 CSV 이름만 바꿉니다. .py 이름은 유지합니다.
```

검사할 CSV만 바꾸기

수정 파일은 work/A16_log_checker.py입니다. **.py 파일 이름은 그대로 둡니다.** 코드 안 source = base / "data" / "bad_readings.csv" 한 줄에서 따옴표 속 CSV 이름만 바꿉니다. 예: source = base / "data" / "readings.csv".

저장 후 course 폴더에서 python -X utf8 work/A16_log_checker.py를 실행합니다. 실행마다 work/check_summary.json과 work/check_issues.json이 덮어써지므로 **결과를 먼저 기록한 뒤** 다음 CSV로 바꿉니다.

따옴표 속 입력 이름	예상 집계·평균	실제 집계·평균	직접 실행/제공 출력 관찰	차이와 근거
readings.csv	——	——	——	——
bad_readings.csv	——	——	——	——
empty_readings.csv	——	——	——	——
missing_readings.csv	——	——	——	——

평균을 낼 숫자가 없다는 결과와 실제 측정 0을 구분해 설명합니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

A17 · 따라 하기와 작성칸

A17 · 정상·빈 입력·잘못된 입력 비교

추가 원문·작업 파일:

- [A17_input.md](#)
- [readings.csv](#)
- [bad_readings.csv](#)
- [empty_readings.csv](#)
- [missing_readings.csv](#)
- [A17_notes.md](#)
- [A16_log_checker.py](#)

목적: 수치가 하나도 없으면 평균 분모가 0입니다. 평균 None은 계산할 수치가 없다는 결과이며 실제 평균 0.0과 의미가 다릅니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)
- [A17_notes.md](#)

단계

1. 네 입력의 기대 집계표를 씁니다.
2. work/A16_log_checker.py 안 source 줄의 따옴표 속 CSV 이름만 바꿔(.py 이름은 유지) 같은 프로그램을 실행합니다.
3. 평균None의 이유와 집계 관계를 확인합니다.
4. 헤더·다른 필드·실제 진위는 미검증으로 기록합니다.

예상 결과: empty 0/0/0/0·None, missing 2/2/2/0·None.

확인 방법: 수치가 없을 때 평균 0을 쓰면 어떤 오해가 생기나요?

오류 해결: 이전 실행 결과 파일을 새 결과로 혼동하지 않도록 입력명·저장·실행 경로를 확인합니다.

검사할 CSV만 바꾸기

수정 파일은 work/A16_log_checker.py입니다. **.py 파일 이름은 그대로 둡니다.** 코드 안 source = base / "data" / "bad_readings.csv" 한 줄에서 따옴표 속 CSV 이름만 바꿉니다. 예: source = base / "data" / "readings.csv".

저장 후 course 폴더에서 python -X utf8 work/A16_log_checker.py를 실행합니다. 실행마다 work/check_summary.json과 work/check_issues.json이 덮어써지므로 **결과를 먼저 기록한 뒤** 다음 CSV로 바꿉니다.

따옴표 속 입력 이름	예상 집계·평균	실제 집계·평균	직접 실행/제공 출력 관찰	차이와 근거
readings.csv	——	——	——	——
bad_readings.csv	——	——	——	——
empty_readings.csv	——	——	——	——
missing_readings.csv	——	——	——	——

평균을 낼 숫자가 없다는 결과와 실제 측정 0을 구분해 설명합니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A18 · 검사기 사용 안내와 근거 전달

개념. 사용 안내는 다음 사람이 같은 입력으로 같은 결과를 확인할 수 있도록 쓰는 문서입니다. 파일·명령·기대 결과·이슈의 뜻·미확인 범위를 함께 전달합니다.

왜 필요한가. 작성자가 없어도 검사 범위를 오해하지 않게 해야 합니다. '검증 완료'보다 온도 변환·범위·미측정 구별을 검사했다고 구체적으로 씁니다.

함께 보는 예. 안내에는 course의 실행 명령, source가 읽는 입력명, work 결과 두 개, 이슈 원문을 확인하는 방법을 씁니다. 확정 근거가 있으면 작업 사본만 고치고 재확인합니다.

헛갈리는 경우. 모든 필드를 검사하거나 실제 센서 정확도를 보장한다고 주장하지 않습니다. 오늘은 로컬 합성 파일의 처리이며 외부 서비스로 전송하지 않습니다.

제공 입력

작성칸: 목적 / 필요한 파일 / 명령 / 입력 선택 / 결과 위치 / 이슈 해석 / 수정 근거 / 미확인 범위.

A18 · 따라 하기와 작성칸

A18 · 검사기 사용 안내와 근거 전달

추가 원문·작업 파일:

- [A18_input.md](#)

- [A18_notes.md](#)

- [usage.md](#)

목적: 작성자가 없어도 검사 범위를 오해하지 않게 해야 합니다. '검증 완료'보다 온도 변환·범위·미측정 구별을 검사했다고 구체적으로 씁니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A18_input.md](#)

- [A18_notes.md](#)

단계

1. work/usage.md에 실행 안내를 씁니다.
2. 짝이 입력 하나의 결과를 설명하게 합니다.
3. 빠진 경로와 계약을 보완합니다.
4. 코드·입력·요약·이슈·사례표를 함께 제출합니다.

예상 결과: 재현 가능한 경로·명령·결과와 검증 범위가 분명한 안내.

확인 방법: 이슈 0건이라는 결과만 남기면 무엇이 부족한가요?

오류 해결: 처음 실행 순서와 오류 확인 순서를 나눠 씁니다. 미확인 사실을 확정하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

용어와 코드 기호 찾아보기

Python 심화 쉬운 용어

용어	뜻
함수	이름 붙여 여러 번 부를 수 있게 묶은 작업
매개변수	함수 정의에서 받을 값에 붙인 이름
인자	함수를 부를 때 실제로 전달하는 값
반환값	함수가 호출한 곳으로 돌려주는 결과
경로	파일의 저장 위치를 나타내는 주소
예외	실행을 계속할 수 없는 상황을 알리는 신호
try/except	작업을 시도하고 특정 오류가 날 때 할 일을 정하는 구조
검증 사례	입력과 기대 결과를 미리 정해 비교하는 한 경우

용어 이름만 외우기보다 활동의 값·파일·출력에서 무엇을 가리키는지 짚습니다. 같은 숫자 모양이라도 의미와 저장 자료형을 구별합니다.

오늘 배운 내용 확인하기

Python 심화 확인과 평가

18활동의 작성·예상·실제 확인을 분리해 봅니다. 속도나 설치 성공만 평가하지 않습니다.

기준	확인할 증거	보완이 필요한 경우
개념	입력·처리·출력 또는 변경 단계의 역할 설명	결과 이름만 외우고 근거를 말하지 못함
수정	지정한 위치와 변경 전후를 연결	여러 곳을 바꾸어 원인을 설명하지 못함
확인	정상·경계·오류 사례의 예상과 실제 구별	실행 성공을 내용 정확성으로 단정

기준	확인할 증거	보완이 필요한 경우
전달	파일·명령·결과·남은 질문을 재현 가능하게 기록	어느 파일을 사용했는지 불명확

각 기준은 ‘독립 설명 가능 / 힌트 후 가능 / 다시 연습’으로 기록합니다. 개인 실행 여부는 별도 체크하며 미실행을 실패한 개념 이해로 간주하지 않습니다.

구간별 확인 질문

1. value는 모든 호출에서 같은 값을 뜻하나요?
2. 20과 30만 확인하면 놓칠 수 있는 실수는 무엇인가요?
3. print와 write_text는 같은 행동인가요?
4. except를 실행하면 실제 온도를 알아낸 것인가요?
5. 미측정도 분모에 넣어 4로 나누면 무엇이 바뀌나요?
6. 이슈 0건이라는 결과만 남기면 무엇이 부족한가요?

제출

work의 활동 노트와 수정 파일, 마지막 종합 작업, 확인한 명령·출력 또는 수동 추적 기록을 제출합니다. 제출 위치는 강사 안내를 따릅니다. 정답 폴더를 학생 결과에 복사하지 않습니다.

2026-10-05 보강 · 새 사례 전이 문제

기존 평가를 보존합니다. 아래 여섯 문제는 복습·도전용이며 설치 성공과 개념 이해를 구별해 평가합니다. 답을 먼저 읽지 말고 근거와 예상값을 기록합니다.

문제 1 · 작업의 입력·처리·결과를 함수 계약으로 쓰기

복습·도전: 물품 라벨에 상품명과 보관함을 받아 ‘장갑 / B02’처럼 한 문자열을 만드는 작업을 설계하세요. 입력 자리 두 개, 반환할 종류, 순서를 바꿨을 때 생길 문제를 문장으로 쓰세요. 첫 입력은 장갑/B02, 둘째 입력은 마스크/A01입니다. 이 두 호출의 결과를 예상하세요.

정답·해설: 입력은 상품명·보관함 문자열, 결과는 두 내용을 순서대로 연결한 문자열입니다. 장갑/B02와 마스크/A01의 결과는 각 입력에 따라 달라집니다. 순서를 바꾸면 B02 / 장갑처럼 업무 양식이 달라집니다. 컴퓨터가 이름의 뜻을 추측해 입력 위치를 고쳐 준다는 답은 오답입니다.

문제 2 · 화면 표시와 다음 계산의 반환값 구별하기

복습·도전: 배송 준비 함수는 포장비 500원을 화면에 표시하지만 반환문이 없습니다. 호출 결과를 fee에 연결한 뒤 총액 3000에 더하려 합니다. 화면값·fee 값·다음 계산을 분리해 설명하고 올바른 역할 배치를 문장으로 설계하세요.

정답·해설: 화면에는 500이 보이지만 fee는 None이며 다음 덧셈은 TypeError가 날 수 있습니다. 함수가 500을 반환하고 호출자가 총액에 더한 뒤 표시하도록 설계합니다. 화면에 500이 보이므로 fee도 500이라는 답은 표시와 반환을 혼동한 것입니다.

문제 3 · 읽은 텍스트에서 값으로, 값에서 저장 파일로

복습·도전: 입력 헤더는 device_id,value이고 한 행은 D09,0입니다. 프로그램은 temperature_c를 찾다가 실패했습니다. 파일 열기·헤더 찾기·숫자 변환 중 어느 단계인지 판단하고 원문을 보존한 수정 계획과 올바른 JSON 수치 표현을 쓰세요.

정답·해설: 헤더 찾기 단계의 문제입니다. 실행 계약이 value를 쓰도록 정해졌다면 접근 키를 value로 맞추되 계약과 원문 헤더를 먼저 확인합니다. 수치 0은 실제 값이므로 JSON에서는 0이며 null이 아닙니다. 주소 수정이나 0 삭제는 현재 증거에 맞는 해결이 아닙니다.

문제 4 · 미측정·잘못된 글자·실제 0의 세 갈래

복습·도전: 수량 칸 원문이 빈 문자열, "0", "여섯", " "입니다. 계약은 정확한 빈 문자열만 미확인, 숫자만 계산 대상으로 정합니다. 네 갈래의 분류와 담당자에게 남길 이슈 원문을 작성하세요.

정답·해설: 빈 문자열은 미확인, "0"은 수치 0, "여섯"과 공백 한 칸은 현재 계약의 숫자 해석 이슈입니다. 두 이슈 원문을 그대로 남깁니다. 공백을 자동 삭제해 미확인으로 합치는 답은 명시되지 않은 정리 규칙을 추가한 것입니다.

문제 5 · 허용 범위의 바깥과 경계를 검사하기

복습·도전: 수업용 배터리 기록의 허용 충전율은 0~100%입니다. -0.1, 0, 50, 100, 100.1의 기대 결과와 바깥을 찾는 논리 문장을 쓰세요. 100.1을 100으로 자동 변경해도 되는지 근거를 설명하세요.

정답·해설: 양 끝 -0.1·100.1은 거부, 0·50·100은 허용입니다. '0 미만 또는 100 초과'가 바깥입니다. 자동 변경은 원문을 바꾸므로 현재 검사 계약만으로는 허용되지 않습니다. and로 쓰면 두 바깥 조건을 동시에 만족하는 값이 없어 오류를 놓칩니다.

문제 6 · 분류 건수와 평균 분모를 보존하기

복습·도전: 원문은 "5", "15", 빈 문자열, "뜨거움", 빈 문자열, "0"입니다. 전체·수치·미측정·이슈·수락 건수와 평균을 구하고 두 검산식을 쓰세요. 수치가 전혀 없는 경우의 평균 표현도 적으세요.

정답·해설: 전체 6, 수치 3, 미측정 2, 이슈 1, 수락 5입니다. $6=5+1$, $5=3+2$ 이고 평균은 $20\div3$ 을 소수 둘째 자리로 반올림한 6.67입니다. 수치가 없으면 None입니다. 수락 5를 분모로 쓰면 4가 되어 미측정을 포함한 잘못된 평균입니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

Python 심화 PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.